



Progetto:

PronuntiApp

Documentazione di Sviluppo

Sviluppo di Mobile Software

a.a. 2023-2024

**Informatica e Tecnologie per la
Produzione del Software**

Realizzato da

Chinni Carella Filippo 764471 ITPS
f.chinnicarella@studenti.uniba.it

Di Gennaro Pasquale 758195 ITPS
p.digennaro5@studenti.uniba.it

Amati Diego 764467 ITPS
d.amati6@studenti.uniba.it

Balzano Nicola 754694 ITPS
n.balzano2@studenti.uniba.it



Dipartimento di Informatica - Università degli studi di Bari

Via Orabona 4 - 70125 - Bari

email: abcduniba@gmail.com

www.uniba.it/it/ricerca/dipartimenti/informatica

Indice

1. Presentazione del Progetto	5
1.1 Introduzione al Progetto	5
1.1.1 Motivazioni Scelta del Caso di Studio	5
1.2 Presentazione della Applicazione	6
1.2.1 Funzionalità della Applicazione	6
1.2.1.1 Feature 1 - Generico: Login	6
1.2.1.2 Feature 2 - Generico: Logout	7
1.2.1.3 Feature 3 - Generico: Registrazione	8
1.2.1.4 Feature 4 - Paziente: Navigazione in uno Scenario di Gioco	9
1.2.1.5 Feature 5 - Paziente: Esercizio Denominazione Immagine	10
1.2.1.6 Feature 6 - Paziente: Esercizio Sequenza Parole	11
1.2.1.7 Feature 7 - Paziente: Esercizio Coppia Immagini	12
1.2.1.8 Feature 8 - Paziente: Selezione del Personaggio	13
1.2.1.9 Feature 9 - Paziente: Sblocco dei Personaggi	14
1.2.1.10 Feature 10 - Paziente: Visualizzazione delle Classifica	15
1.2.1.11 Feature 11 - Genitore: Monitoraggio della Terapia	16
1.2.1.12 Feature 12 - Genitore: Visualizzazione degli Appuntamenti	17
1.2.1.13 Feature 13 - Genitore: Visualizzazione e Modifica del Profilo	17
1.2.1.14 Feature 14 - Logopedista: Visualizzazione della Lista Pazienti	18
1.2.1.15 Feature 15 - Logopedista: Creazione di Terapie	19
1.2.1.16 Feature 16 - Logopedista: Monitoraggio della Terapia	20
1.2.1.17 Feature 17 - Logopedista: Creazione e Visualizzazione Appuntamenti	21
1.2.1.18 Feature 18 - Logopedista: Visualizzazione e Modifica del Profilo	23
1.2.2 Tecnologie di Sviluppo Adottate	24
1.2.2.1 Sviluppo Applicazione: Android Studio	24
1.2.2.2 Tool di Versioning: Git (+ GitHub)	24
1.2.2.3 Database - Dati Testuali: Firebase Realtime Database	24
1.2.2.4 Database - Dati Multimediali: Firebase Storage	24
1.2.2.5 Autenticazione: Firebase Authentication	25
1.2.2.6 API Riconoscimento Vocale: Google Cloud Speech-to-Text API	25
1.2.2.7 API Conversione Audio: FFmpeg	25
1.2.2.8 Libreria per Visualizzazione di Immagini: Picasso	26
1.2.2.9 Libreria per Visualizzazione di Immagini: Glide	26
1.2.2.10 Realizzazione Icona di Prodotto: Figma	26
1.2.3 Limiti della Applicazione	26
1.2.4 Difficoltà Principali dello Sviluppo	27
1.2.4.1 Utilizzo di Database NoSQL	27

1.2.4.2 Asincronicità delle Operazioni su Database	30
1.2.4.3 Asincronicità di alcune Operazioni Android	32
1.2.4.4 Classi ViewModel	33
1.2.4.5 Tempistiche del Progetto	35
2. Architettura del Sistema	38
2.1 Architettura di Alto Livello del Sistema	38
2.2 Diagramma delle Classi	39
2.3 Gerarchia dei Dati	39
2.3.1 Cruscotto Firebase	40
3. Ruoli del Team	43
3.1 Struttura del Team	43
3.2 Suddivisione	43
3.2.1 Filippo Chinni Carella	43
3.2.2 Pasquale Di Gennaro	46
3.2.3 Diego Amati	47
3.2.4 Nicola Balzano	47

PRESENTAZIONE DEL PROGETTO

PRONUNTIAPP

1. PRESENTAZIONE DEL PROGETTO

1.1 Introduzione al Progetto

La seguente relazione riguarda la Documentazione di Sviluppo del gruppo ABCD riguardo al Caso di Studio: PronuntiApp.

La seguente relazione descrive le Funzionalità realizzate della Applicazione, correlate con Tecnologie Adottate, Difficoltà nello Sviluppo e Architettura del Sistema.

Inoltre si include una descrizione dell'operato di ogni singolo membro del gruppo.



1.1.1 Motivazioni Scelta del Caso di Studio

Il Caso di studio PronuntiApp è stato preferito in quanto si è ritenuto più interessante l'aspetto della Gamification.

Il concetto di Gamification negli anni è diventato sempre più preponderante nel mondo della UX, molti sistemi progettano la propria interazione utilizzando nozioni di Gamification.

PronuntiApp ha l'obiettivo di far seguire ai bambini che la utilizzano una terapia di Logopedia. Essendo il target di utilizzatori di età molto bassa, serve che l'App invogli il bambino a svolgere le attività facenti parte della terapia.

PronuntiApp fa questo mediante la Gamification, le attività sono proposte sotto forma di gioco in modo da invogliare il bambino nello svolgimento. Si è ritenuto questo aspetto particolarmente interessante.

In particolare il concetto di scenario di gioco e oggetti grafici draggabili sullo schermo, ha catturato l'attenzione del gruppo, spingendo alla volontà di cimentarsi direttamente.

1.2 Presentazione della Applicazione

PronuntiApp è una App per la terapia a distanza rivolta a utenti con disturbi del linguaggio, che li aiuti nello svolgimento di esercizi logopedici utilizzando meccaniche di Gamification.

L'Applicazione si riferisce a 3 tipologie di utenti (attori): Logopedisti, Genitori e Pazienti.

Pazienti:

I Pazienti sono il vero fulcro dell'App, l'App esiste per loro e sono i principali utilizzatori.

Con "paziente" si intende un Bambino, in quanto l'App ha l'obiettivo di fare seguire una terapia di Logopedia per un target molto giovane.

I bambini svolgono la Terapia assegnatagli sotto forma di gioco, le attività della terapia sono di fatto dei minigiochi che il bambino deve eseguire.

Logopedista:

I Logopedisti sono coloro che creano e gestiscono la Terapia per i bambini.

Il principale utilizzo della App da parte dei Logopedisti è quello di assegnare terapie e esercizi ai bambini e stabilire un contatto con i Genitori dei Bambini.

Genitore:

I Genitori sono i genitori dei Bambini che utilizzano l'App.

L'utilizzo della App da parte del Genitore è diverso dal suo bambino; il principale utilizzo è monitorare i progressi del Bambino nel procedimento della terapia e fissare appuntamenti con il Logopedista.

1.2.1 Funzionalità della Applicazione

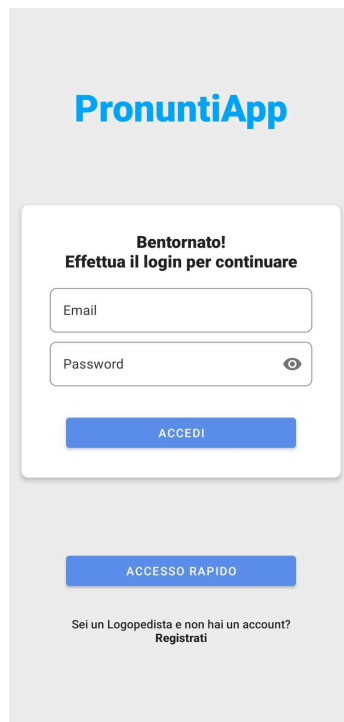
Descrizione delle Funzionalità dell'Applicazione, divise per tipologia di Utente.

1.2.1.1 Feature 1 - Generico: Login

Funzionalità che permette all'utente di Loggarsi nell'applicazione.

Le informazioni di login vengono salvate dall'app, quindi anche se chiude l'app, quando la si riapre non sarà necessario rieffettuare il Login.

Il login è valido per tutte e 3 le tipologie di utente, una volta inserita email e password, l'utente sarà portato alla sezione home relativa alla sua tipologia di profilo.



Dettagli di Implementazione:

Per realizzare il login (o meglio il login effettivo), si è utilizzato Firebase Authentication.

Data una email e una password, Firebase Authentication (se tale email è registrata sulla piattaforma) è in grado di eseguire il login di tale utente nell'app e di ritornare uno UserId.

Lo UserId è utilizzato per fare delle query al db che restituiscono la tipologia dell'utente che ha effettuato il login e la struttura dati associata a tale utente.

Infine viene inizializzata la Activity relativa a tale utente con i dati presi in precedenza.

Per quanto riguarda il salvataggio di email e password al fine di login futuri dopo la chiusura dell'app, si sono utilizzate le Shared Preferences.

1.2.1.2 Feature 2 - Generico: Logout

Funzionalità che permette all'utente di uscire dall'account nell'applicazione.

Siccome le informazioni riguardanti il login vengono salvate, allora è necessaria una funzionalità per uscire dall'account, in modo da poter eseguire il login con un altro account.

Dettagli di Implementazione:

Si è utilizzato Firebase Authentication per eseguire il logout lato Firebase Authentication.

Per il Logout “lato locale” semplicemente si eliminano i dati riguardanti email e password dalle Shared Preferences.

1.2.1.3 Feature 3 - Generico: Registrazione

Funzionalità che permette a un nuovo utente di registrarsi sulla Applicazione, in modo da poterla utilizzare e da poter entrarci in futuro.

La Registrazione dei Logopedisti è accessibile dalla prima schermata dell'app, ovvero quella di Login, quindi i Logopedisti possono creare account a piacimento nella applicazione.

La Registrazione di Genitori e Pazienti è invece a carico del logopedista, che dalla lista dei suoi Paziente può cliccare su “Aggiungi Paziente” per registrare all'app un Paziente e un Genitore ad esso relativo.

The image displays three mobile application screens for registration:

- Screen 1: Registrazione (PronuntiApp)**
Title: Benvenuto Dottore!
Subtitle: Inserisci i tuoi dati per iniziare ad usare l'applicazione
Fields: Nome, Cognome, Email, Username, Telefono, Indirizzo, Password, Conferma Password
Button: REGISTRATI
- Screen 2: Registrazione paziente e genitore**
Title: Ciao Dottore.
Subtitle: Inserisci qui i dati del nuovo Paziente
Section: Informazioni Paziente
Fields: Nome, Cognome, Email, Username, Et , Sesso, Data di nascita, Password, Conferma Password
Bottom navigation: Pazienti, Appuntamenti, Profilo
- Screen 3: Registrazione genitore**
Fields: Data di nascita, Password, Conferma Password
Section: Informazioni Genitore
Fields: Nome, Cognome, Email, Username, Telefono, Password, Conferma Password
Button: REGISTRA PAZIENTE
Bottom navigation: Pazienti, Appuntamenti, Profilo

Dettagli di Implementazione:

Per la registrazione degli utenti è stato utilizzato Firebase Authentication, che permette di registrare degli utenti lato Firebase Authentication con l'email e la password fornite, sempre che l'email non sia già in uso.

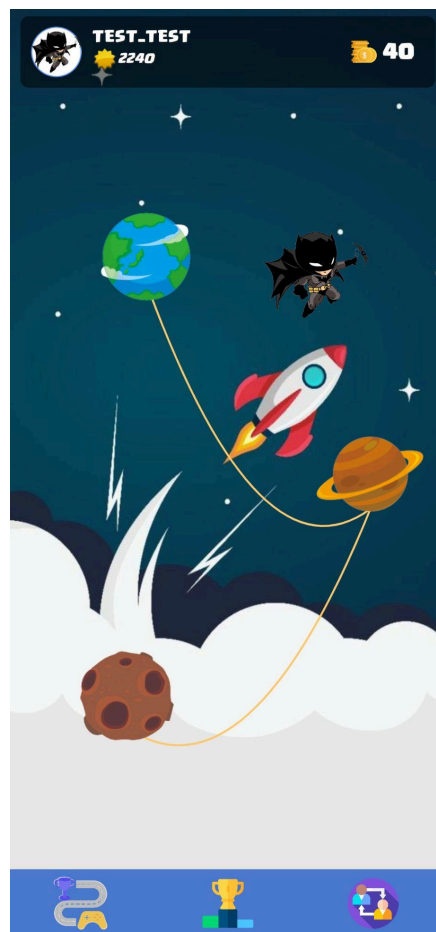
Lato Database invece, una volta avvenuta la registrazione lato Firebase Authentication, viene restituito uno UserId, che viene utilizzato come id dell'utente nel database. Inoltre l'oggetto utente (che può essere un Logopedista, Genitore o Paziente) viene inizializzato con i dati forniti al momento della registrazione (e anche con altri dati di default nel caso del paziente).

1.2.1.4 Feature 4 - Paziente: Navigazione in uno Scenario di Gioco

Gli esercizi della Terapia vengono rappresentati sotto forma di “tappe” in uno scenario di gioco interattivo. Su tale scenario di gioco è presente un personaggio, trascinabile su tutto lo scenario di gioco.

Il bambino (paziente) per accedere agli esercizi deve spostare il personaggio all'interno dello Scenario, e portarlo su una delle tappe; al che, attraverso un feedback sia grafico (ingrandimento dell'immagine che rappresenta la tappa), che tattile (vibrazione) capirà che lasciando andare il personaggio, si attiverà un esercizio.

Gli scenari sono sempre composti da 3 esercizi, in genere tutti di tipologia diversa. Le tipologie di esercizi sono 3: Esercizio Denominazione Immagine, Esercizio Sequenza Parole, Esercizio Coppia Immagini. Si parlerà meglio di ciascuno dopo.



Dettagli di Implementazione:

Per quanto riguarda la vibrazione all'avvicinamento dei personaggi alle tappe, si è utilizzato un SystemService chiamato Vibrator.

Per quanto riguarda il dragging dell'oggetto grafico rappresentante il personaggio all'interno dello scenario, si è utilizzato sulla view un listener di tipo

“onTouchListener”, su cui si eseguivano operazioni per la modifica delle coordinate del personaggio nell’area dello scenario.

Per quanto riguarda l’attivazione dell’esercizio all’avvicinamento del personaggio, si è implementata una meccanica simile alle collisioni (ovvero si verifica se la hitbox del personaggio è all’interno della hitbox della tappa).

1.2.1.5 Feature 5 - Paziente: Esercizio Denominazione Immagine

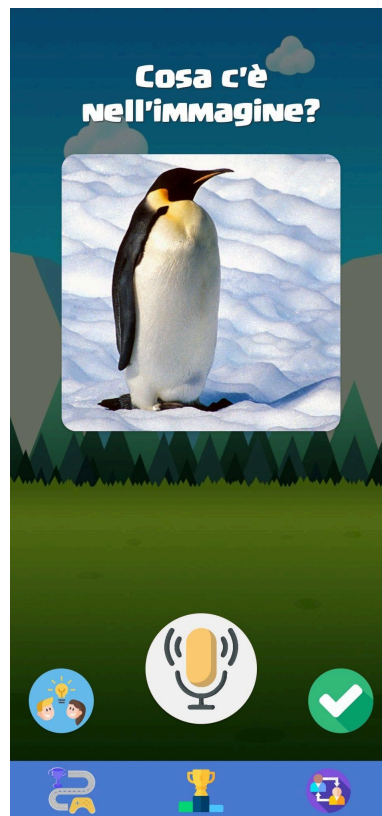
Questa feature rappresenta l’esercizio di tipo Denominazione Immagine.

Il Paziente visualizza sullo schermo una immagine. Dovrà registrare un audio dove pronuncia la parola che quella immagine rappresenta.

Il paziente ha la possibilità di ascoltare fino a 3 aiuti, dove l’aiuto è semplicemente la riproduzione dell’audio della parola che dovrà dire.

Il paziente ha inoltre la possibilità di sovrascrivere la sua registrazione.

Per confermare il completamento dell’esercizio deve cliccare sul tasto di conferma.



Dettagli di Implementazione:

Per registrare gli audio si è utilizzato AudioRecorder, una astrazione dell'AudioRecord di Android nativo.

Per riprodurre gli audio si è utilizzato AudioPlayer, una astrazione del MediaPlayer di Android nativo.

Per il riconoscimento della parola pronunciata dal paziente, si è utilizzata l'API di Google Cloud: Speech-To-Text API.

Per controllare la corretta esecuzione dell'esercizio si comparava quanto ritornato dall'API con quella che è la parola effettiva rappresentata sull'immagine.

1.2.1.6 Feature 6 - Paziente: Esercizio Sequenza Parole

Questa feature rappresenta l'esercizio di tipo Sequenza Parole.

Il Paziente dovrà avviare la riproduzione di un Audio, all'interno del quale vengono pronunciate 3 parole.

Il Paziente dovrà poi ripetere la sequenza di parole ascoltata, nello stesso ordine.

Per confermare il completamento dell'esercizio deve cliccare sul tasto di conferma.



Dettagli di Implementazione:

Per registrare gli audio si è utilizzato AudioRecorder, una astrazione dell'AudioRecord di Android nativo.

Per riprodurre gli audio si è utilizzato AudioPlayer, una astrazione del MediaPlayer di Android nativo.

Per il riconoscimento delle parole pronunciate dal paziente, si è utilizzata l'API di Google Cloud: Speech-To-Text API.

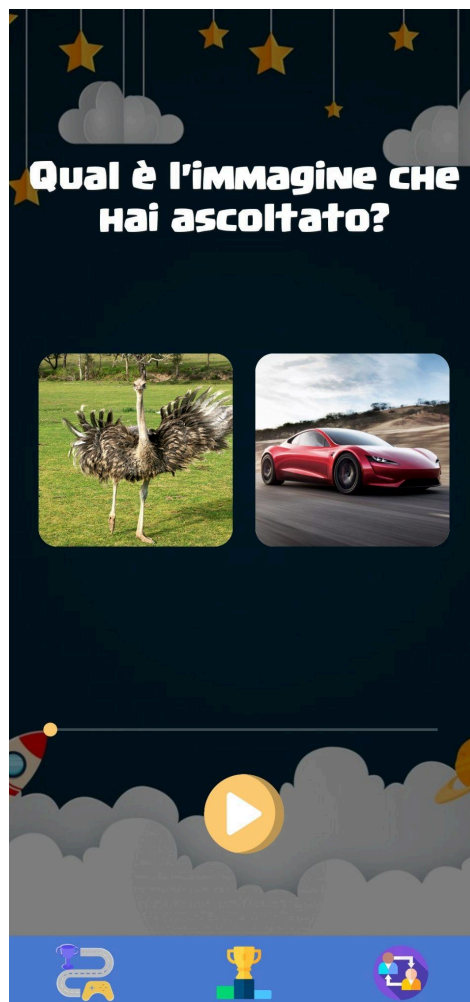
Per controllare la corretta esecuzione dell'esercizio si comparava quanto ritornato dall'API con quella che è la effettiva sequenza di parole da ripetere.

1.2.1.7 Feature 7 - Paziente: Esercizio Coppia Immagini

Questa feature rappresenta l'esercizio di tipo Coppia Immagini.

Il Paziente deve ascoltare la riproduzione di un audio dove viene pronunciata una parola.

Il Paziente dovrà poi selezionare, tra le due immagini presenti, quale delle due rappresenta la parola che ha ascoltato.



Dettagli di Implementazione:

Per riprodurre gli audio si è utilizzato MediaPlayer, una astrazione del MediaPlayer di Android nativo.

Per generare randomicità, si è utilizzata una semplice funzione di random nativa di Java.

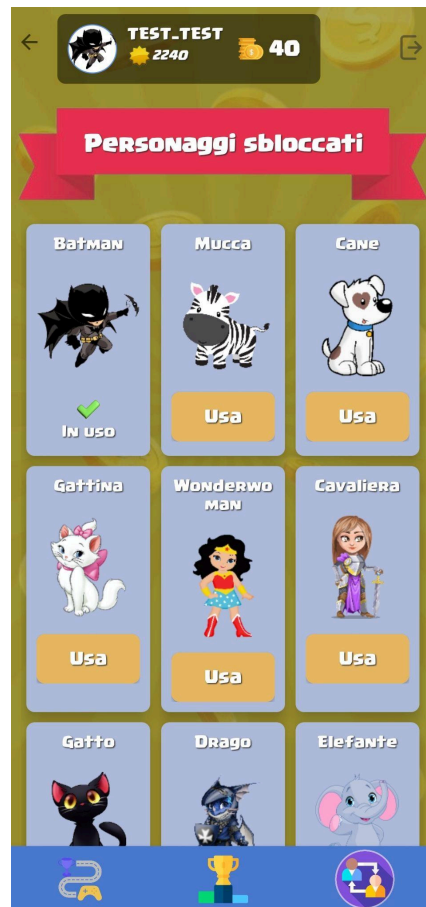
Il numero generato definisce il posizionamento dell'immagine corretta e della immagine errata.

1.2.1.8 Feature 8 - Paziente: Selezione del Personaggio

Funzionalità che permette al Paziente di cambiare il personaggio che deve essere visualizzato sullo scenario di gioco.

L'utente può selezionare i personaggi dalla lista di quelli sbloccati.

Lo sblocco di nuovi personaggi viene trattato in un'altra feature.



Dettagli di Implementazione:

A livello della struttura dati del Paziente, vi è una mappa dove sono mappati id del personaggio e status, ovvero se quel personaggio è bloccato, sbloccato o selezionato.

Ad ogni avvio dell'App, viene eseguito il retrieve dei personaggi effettivi dal Database; al che per generare la lista dei personaggi per il paziente, verrà generato

per ogni personaggio una entry che ne descrive lo status (se è bloccato, sbloccato o selezionato).

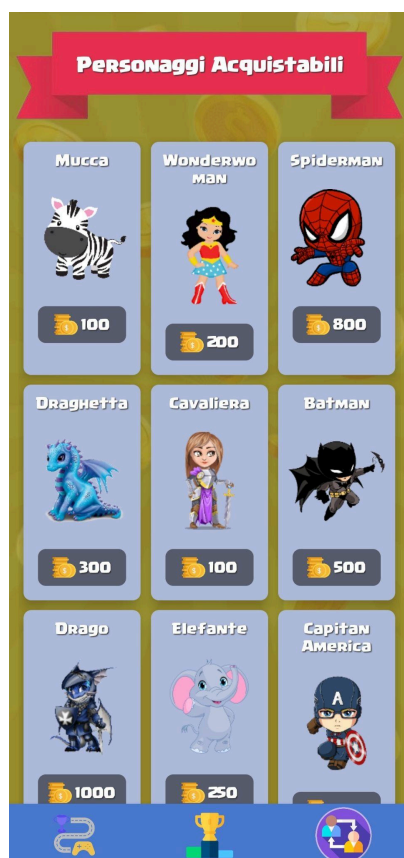
I cambiamenti di status vengono salvati, in modo che al prossimo avvio il paziente abbia lo stesso personaggio selezionato in precedenza.

1.2.1.9 Feature 9 - Paziente: Sblocco dei Personaggi

Funzionalità che permette al Paziente di sbloccare nuovi personaggi utilizzando la valuta di gioco che guadagna eseguendo gli esercizi.

Selezionando un personaggio nella lista dei personaggi acquistabili, se il Paziente ha abbastanza valuta per acquistare il personaggio selezionato, allora può sbloccarlo.

Allo sblocco il personaggio appena acquistato viene posto come personaggio selezionato.



Dettagli di Implementazione:

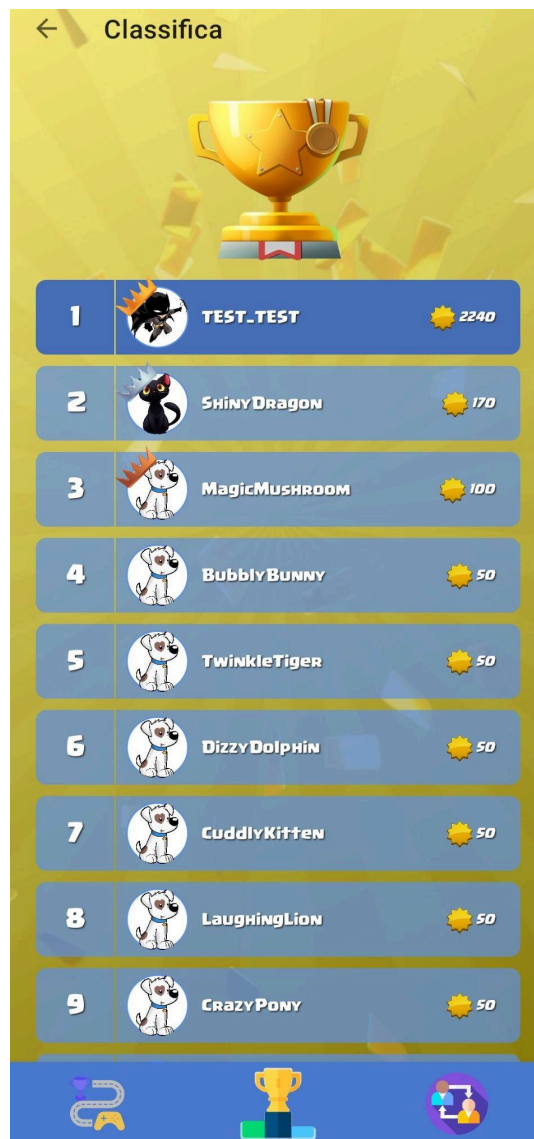
Per quanto riguarda il check sulla possibilità di sbloccare un personaggio, nel caso in cui la valuta del paziente non sia sufficiente per sbloccare il personaggio desiderato, viene visualizzato un dialogo che fa capire all'utente di non avere abbastanza monete.

1.2.1.10 Feature 10 - Paziente: Visualizzazione delle Classifica

Funzionalità che permette al paziente di vedere la classifica di tutti i pazienti presenti del suo logopedista.

I pazienti sono classificati in base al Punteggio Totale, ovvero la somma di tutte la valuta accumulata durante l'esecuzione degli esercizi di tutte le terapie.

Il paziente che visualizza la classifica è evidenziato rispetto agli altri nella classifica.



Dettagli di Implementazione:

Per realizzare ogni entry della classifica sono stati utilizzati degli Adapter, e creato ad-hoc un oggetto Paziente-Custom, che combina i dati del paziente come Username e Punteggio Totale, e la texture del suo personaggio selezionato.

1.2.1.11 Feature 11 - Genitore: Monitoraggio della Terapia

Funzionalità che permette al Genitore di monitorare i progressi di svolgimento degli esercizi della terapia da parte del paziente (ovvero del suo bambino).

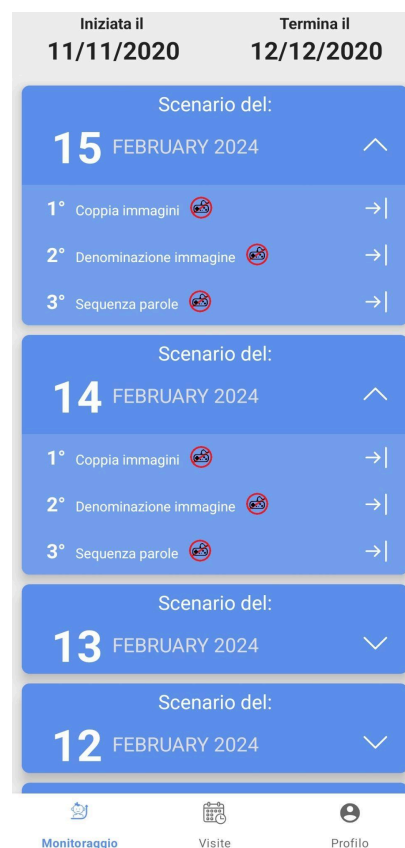
Il genitore ha la possibilità innanzitutto di navigare tra le Terapie, tra quelle passate e attuale, e visualizzare, per ogni Scenario di gioco, il risultato dei singoli esercizi.

Nella schermata principale del monitoraggio vede solo una panoramica generale della esecuzione dell'esercizio (ovvero se è stato completato bene, completato in maniera errata, o non eseguito).

Cliccando sul singolo esercizio, il genitore visualizza per intero l'esercizio che il bambino doveva svolgere e visualizza l'esito di tale esercizio.

Se l'esercizio è di una delle 2 tipologie che prevede la registrazione di un audio, allora il genitore può riascoltare l'audio registrato dal bambino.

Il genitore ha anche la possibilità di cambiare la data di inizio di uno scenario.



Dettagli di Implementazione:

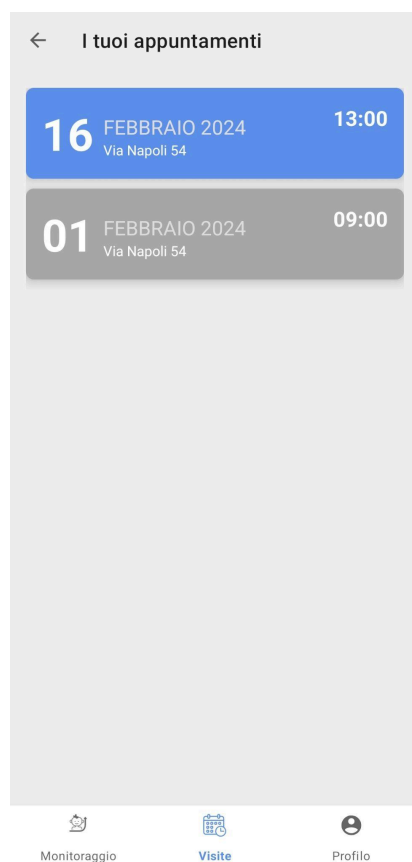
Per la visualizzazione “annidata” dei vari elementi si sono utilizzati Adapter lato Java e “item” lato XML, dove gli item sono o Fragment oppure View (in genere ViewGroup) complesse.

1.2.1.12 Feature 12 - Genitore: Visualizzazione degli Appuntamenti

Funzionalità che permette al genitore di visualizzare gli appuntamenti, passati e futuri, con il logopedista.

Viene visualizzata data, ora e luogo per ogni Appuntamento. Gli appuntamenti passati vengono visualizzati in grigio, quelli futuri in blu.

Il genitore non può creare, modificare o rimuovere appuntamenti, ma solo il Logopedista può farlo.



Dettagli di Implementazione:

Nessun dettaglio degno di nota sulla implementazione da descrivere qui.

1.2.1.13 Feature 13 - Genitore: Visualizzazione e Modifica del Profilo

Funzionalità che permette al genitore di visualizzare e modificare i dati del proprio profilo e del profilo del proprio bambino.

Per entrare nella "modalità di modifica" il genitore deve cliccare il tasto "modifica profilo" e ricliccarlo per confermare le modifiche fatte.

Il genitore può solo modificare il numero di telefono (oltre alla immagine).



Dettagli di Implementazione:

L'immagine di profilo non è realmente implementata ma è solo simulata.

Infatti nella modalità di modifica è possibile inserire o modificare una immagine tramite File Picker ma questa non viene salvata, e viene persa se si cambia Fragment.

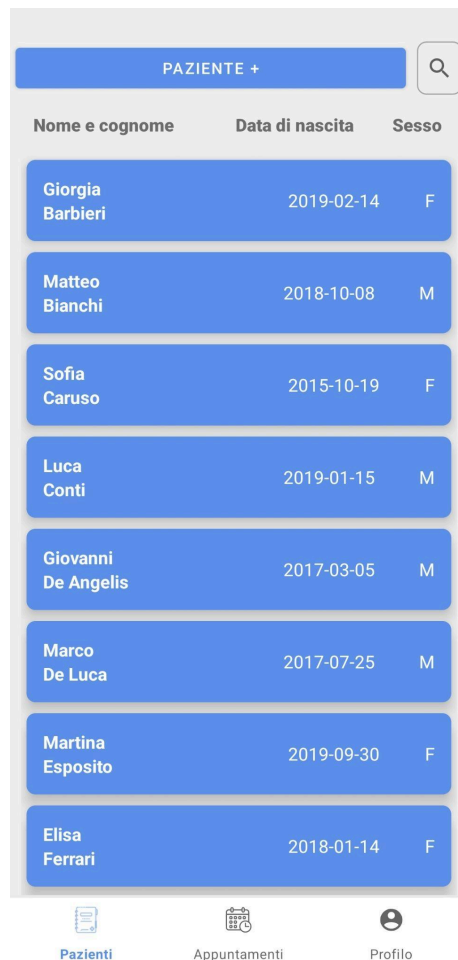
1.2.1.14 Feature 14 - Logopedista: Visualizzazione della Lista Pazienti

Funzionalità che permette al Logopedista di visualizzare la lista dei propri pazienti.

Cliccando sul paziente, il Logopedista può entrare nell'area di monitoraggio delle terapie relative a quel paziente.

Il Logopedista può aggiungere un nuovo paziente cliccando sul tasto "Paziente +".

Per aggiungere un nuovo paziente, il logopedista verrà portato alla funzionalità di registrazione relativa a pazienti e genitori. Qui inserirà i loro dati per poter registrare il nuovo paziente.



Nome e cognome	Data di nascita	Sesso
Giorgia Barbieri	2019-02-14	F
Matteo Bianchi	2018-10-08	M
Sofia Caruso	2015-10-19	F
Luca Conti	2019-01-15	M
Giovanni De Angelis	2017-03-05	M
Marco De Luca	2017-07-25	M
Martina Esposito	2019-09-30	F
Elisa Ferrari	2018-01-14	F

Dettagli di Implementazione:

Sono stati utilizzati Adapter e RecyclerView per la visualizzazione dinamica della lista.

Per l'aggiornamento dinamico, sono stati utilizzati i LiveData, legati alle strutture dati definite nel ViewModel della activity corrente.

[Questo Dettaglio di Implementazione è simile e comune per tutte le funzionalità che prevedono la visualizzazione di una lista di elementi composti]

1.2.1.15 Feature 15 - Logopedista: Creazione di Terapie

Funzionalità che permette al Logopedista di creare nuove Terapie per il paziente selezionato.

Cliccando su nuova Terapia, il logopedista può inserire i dati relativi alla nuova terapia.

Per prima cosa inserisce le informazioni riguardanti la Terapia in sé.

Poi inserisce, uno ad uno, gli Scenari di quella terapia. Poi inserisce, uno ad uno, gli Esercizi per ogni Scenario.

Il Logopedista, ad ogni aggiunta di Scenario o Esercizio può decidere se selezionare uno Scenario o Esercizio da una lista di template predefiniti, oppure creare lo Scenario o Esercizio da 0.

Per gli Scenari dovrà fornire le immagini che compongono lo scenario.

Per gli Esercizi dovrà fornire immagini e registrare audio per comporre l'esercizio; ovviamente ogni esercizio sarà composto in modo diverso a seconda della tipologia.

Dettagli di Implementazione:

Si sono utilizzati gli oggetti Bundle dei vari fragment per lo scambio di informazioni tra i vari passaggi della creazione della terapia.

1.2.1.16 Feature 16 - Logopedista: Monitoraggio della Terapia

Funzionalità che permette al Logopedista di monitorare i progressi di svolgimento degli esercizi della terapia da parte di un paziente.

Il logopedista ha la possibilità innanzitutto di navigare tra le Terapie, tra quelle passate e attuale, e visualizzare, per ogni Scenario di gioco, il risultato dei singoli esercizi.

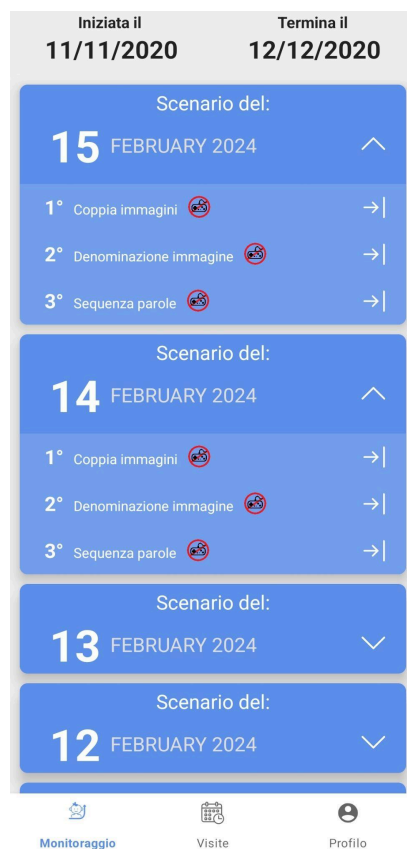
Nella schermata principale del monitoraggio vede solo una panoramica generale della esecuzione dell'esercizio (ovvero se è stato completato bene, completato in maniera errata, o non eseguito).

Cliccando sul singolo esercizio, il logopedista visualizza per intero l'esercizio che il paziente doveva svolgere e visualizza l'esito di tale esercizio.

Se l'esercizio è di una delle 2 tipologie che prevede la registrazione di un audio, allora il logopedista può riascoltare l'audio registrato dal paziente.

Il logopedista ha anche la possibilità di cambiare la data di inizio di uno scenario.

Insomma è una funzionalità praticamente uguale alla sua versione nel lato genitore della applicazione.



Dettagli di Implementazione:

Per la visualizzazione “annidata” dei vari elementi si sono utilizzati Adapter lato Java e “item” lato XML, dove gli item sono o Fragment oppure View (in genere ViewGroup) complesse.

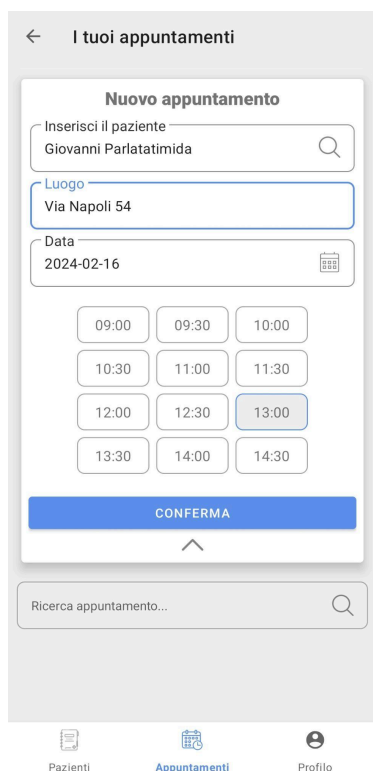
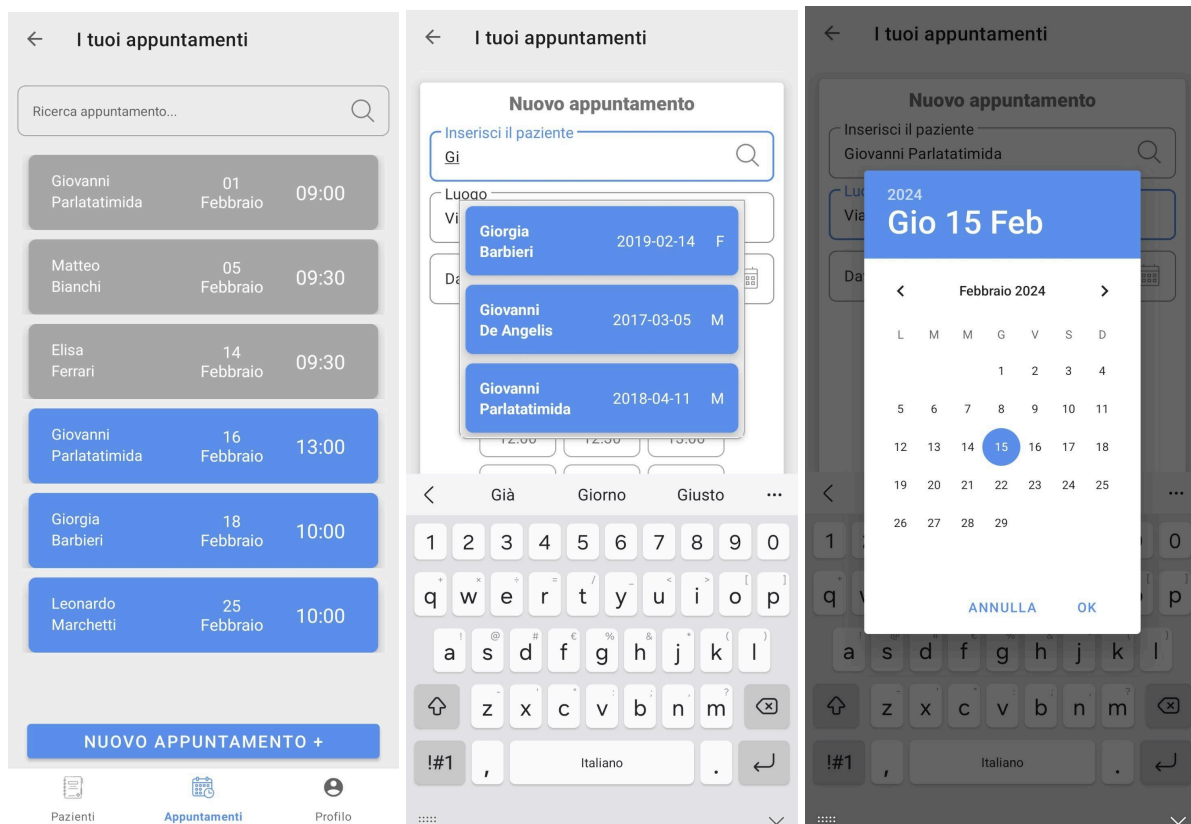
1.2.1.17 Feature 17 - Logopedista: Creazione e Visualizzazione Appuntamenti

Funzionalità che permette al Logopedista di visualizzare e modificare la lista dei suoi appuntamenti.

Per ogni appuntamento vengono visualizzate Data, Ora, Luogo e Nome e Cognome del paziente cui è riferito l'esercizio.

Il Logopedista può creare un appuntamento inserendo i dati prima citati. Per facilitare la creazione, può cercare un paziente tramite la barra di inserimento, che include dei suggerimenti mentre la si compila. Per la selezione della data, è disponibile un Date Picker.

Cliccando su un Appuntamento nella lista degli appuntamenti, è visualizzabile il luogo, e la possibilità di rimuovere l'appuntamento dalla lista degli appuntamenti.



Dettagli di Implementazione:

Per quanto riguarda la selezione della Data è stato implementato un DatePicker; il quale utilizza uno dei DatePicker predefiniti di Android.

Per quanto riguarda la selezione dell'orario, piuttosto che utilizzare un TimePicker si è utilizzato un sistema di selezione della fascia oraria.

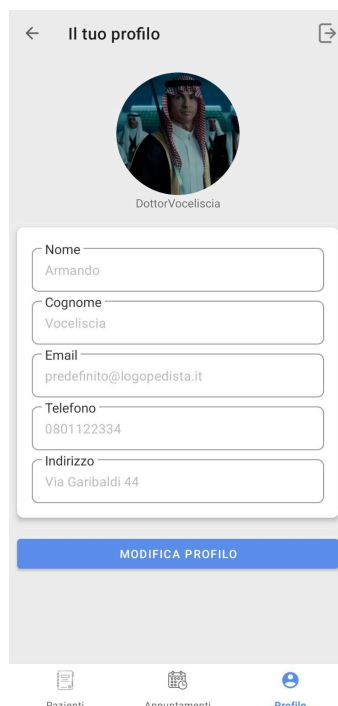
Per quanto riguarda l'implementazione delle barre di ricerca, non si è utilizzato nulla di complicato, semplicemente un adapter prende in input una lista di pazienti e viene aggiornato dinamicamente in base al nome o cognome.

1.2.1.18 Feature 18 - Logopedista: Visualizzazione e Modifica del Profilo

Funzionalità che permette al logopedista di visualizzare e modificare i dati del proprio profilo.

Per entrare nella “modalità di modifica” il logopedista deve cliccare il tasto “modifica profilo” e ricliccarlo per confermare le modifiche fatte.

Il logopedista può solo modificare il numero di telefono e l'indirizzo (oltre alla immagine).



Dettagli di Implementazione:

L'immagine di profilo non è realmente implementata ma è solo simulata.

Infatti nella modalità di modifica è possibile inserire o modificare una immagine tramite File Picker ma questa non viene salvata, e viene persa se si cambia Fragment.

1.2.2 Tecnologie di Sviluppo Adottate

Descrizione di Tecnologie, strumenti e librerie adottati per lo sviluppo della App.

1.2.2.1 Sviluppo Applicazione: Android Studio

La Tecnologia principale di sviluppo adottata è ovviamente Android Studio.

Android Studio è l'IDE più utilizzato per programmare una Applicazione Android a basso livello.

1.2.2.2 Tool di Versioning: Git (+ GitHub)

Per il Versioning del codice di progetto della applicazione si è utilizzato Git.

Git è uno strumento a linea di comando per il Version Control dei progetti software. In particolare si è utilizzato Git in modo integrato ad Android Studio, il quale offre anche un collegamento automatico a GitHub, per distribuire il codice di progetto anche su Repository remoto e consentire quindi il lavoro a distanza dei membri del team.

1.2.2.3 Database - Dati Testuali: Firebase Realtime Database

Per la persistenza dei dati è stato utilizzato Firebase. In particolare Firebase mette a disposizione due tipi diversi di database, Firestore e Realtime Database; per il progetto è stato utilizzato Firebase Realtime Database.

Firebase Realtime Database è un Database online NoSQL di Google.

I dati sono strutturati in maniera gerarchica, nello specifico, tutti i nodi sono un unico grande file in formato JSON.

FirebaseDatabase è la classe in Java che permette di interagire con il Database, fornendo dei metodi per Create, Read, Update, Delete dei dati.

Firebase permette di gestire Firebase Realtime Database, Firebase Storage e Firebase Authentication nello stesso progetto Firebase.

Documentazione: [Firebase Realtime Database](#)

1.2.2.4 Database - Dati Multimediali: Firebase Storage

Per la persistenza dei dati multimediali (immagini e audio) è stato utilizzato Firebase Storage.

Firebase Storage è in pratica un repository per file multimediali, possono essere create cartelle per la categorizzazione dei file.

FirebaseStorage è la classe in Java che permette di interagire con lo Storage, fornendo dei metodi per Create, Read, Update, Delete dei dati.

Firebase permette di gestire Firebase Realtime Database, Firebase Storage e Firebase Authentication nello stesso progetto Firebase.

Documentazione: [Firebase Storage](#)

1.2.2.5 Autenticazione: Firebase Authentication

Per agevolare il processo di Autenticazione (Login e Registrazione) degli utenti sulla Applicazione, è stato utilizzato Firebase Authentication.

Firebase Authentication permette di registrare utenti a partire da una email e una password, ma permette anche di impostare metodi di accesso alternativi (come Google, Facebook, ecc).

FirebaseAuth è la classe in Java che fornisce metodi per il login e la registrazione dell'utente, e l'ottenimento dell'utente corrente e del suo id.

Firebase permette di gestire Firebase Realtime Database, Firebase Storage e Firebase Authentication nello stesso progetto Firebase.

Documentazione: [Firebase Authentication](#)

1.2.2.6 API Riconoscimento Vocale: Google Cloud Speech-to-Text API

Per la verifica della corretta esecuzione di un esercizio di tipo Riconoscimento Denominazione Immagine e Ripetizione Sequenza Parole è stata utilizzata la libreria Google Cloud Speech-to-text-API.

Google Cloud Speech-to-text-API permette di riconoscere le parole pronunciate in una registrazione salvata su un file, restituendo una lista di String contenente le parole riconosciute.

Per funzionare Google Cloud Speech-to-Text API necessita di alcune altre librerie (dipendenze): [google-cloud-core](#), [Google Cloud BOM](#), [gRPC](#).

Documentazione: [Google Cloud Speech-to-Text API](#)

1.2.2.7 API Conversione Audio: FFmpeg

Data l'ottimizzazione della libreria Google Cloud Speech-to-text-API nel riconoscimento di audio con codec LINEAR 16 a 16000 bit/secondo (Bitrate), la registrazione dell'audio tramite microfono nell'applicazione avviene direttamente con tale campionamento e codec.

Da notare però è che, i file audio con codec LINEAR 16, non risultano riproducibili sulla maggior parte dei media player comuni dei dispositivi più diffusi.

Dovendo essere gli audio salvati per scopi futuri, si è ritenuto importante dare all'utente la possibilità di riprodurre gli audio registrati nell'applicazione anche all'esterno di essa.

Per rendere possibile ciò, si è considerata la conversione dell'audio registrato da codec LINEAR 16 a codec libmp3lame associato all'estensione audio mp3.

Questa conversione è stata effettuata utilizzando la libreria FFmpegKit.

FFmpegKit è una libreria wrapper Java che fornisce un'interfaccia per utilizzare FFmpeg su diverse piattaforme come Android e altre. FFmpeg è uno strumento open-source ampiamente utilizzato per l'elaborazione di audio e video.

Documentazione: [FFmpeg](#)

1.2.2.8 Libreria per Visualizzazione di Immagini: Picasso

Per il caricamento dinamico delle immagini all'interno dei singoli widget grafici (es. ImageView) è stata scelta la libreria Picasso, che ha reso semplice la gestione, il caricamento e la visualizzazione delle immagini dal Google Firebase Storage.

A differenza di Glide, Picasso offre funzionalità molto basilari per la visualizzazione delle immagini

In ogni caso Picasso e Glide sono state utilizzate praticamente per gli stessi scopi all'interno del progetto.

Documentazione: [Picasso](#)

1.2.2.9 Libreria per Visualizzazione di Immagini: Glide

Glide è una versione più avanzata, completa e complessa di Picasso.

Offre le stesse funzionalità di base di Picasso, e in più offre funzionalità aggiuntive come il coordinamento con il ciclo di vita dell'Activity, o la visualizzazione di immagini complesse come le GIF.

In ogni caso Picasso e Glide sono state utilizzate praticamente per gli stessi scopi all'interno del progetto.

Documentazione: [Glide](#)

1.2.2.10 Realizzazione Icona di Prodotto: Figma

Figma è un Design Tool online utilizzato per design di interfacce, prototipazione e sviluppo frontend.

Figma non nasce nativamente come strumento per la realizzazione di icone, ma mette a disposizione strumenti come: modifica di SVG, visualizzazione di griglia, colori e riempimento, generazione delle ombre con definizione di forza e direzione delle stesse.

Questo insieme di tool rendono Figma ottimo per la progettazione di una Icona di Prodotto.

1.2.3 Limiti della Applicazione

Alcuni limite della Applicazione rispetto a quelle che erano alcune aspettative iniziali.

Genitori con più Bambini:

Al momento attuale, un Genitore non può avere più Pazienti (bambini).

Con lo schema dei dati originale, questa possibilità era contemplata, infatti nella relazione tra genitore e paziente, l'entità "più forte" era il genitore, sia lato classi che lato database.

Con l'evoluzione del progetto, e in particolare in relazione alle difficoltà riscontrate sullo sviluppo del Database (meglio approfondite nella sezione apposita), il Paziente è infine diventato contenitore di Genitore.

(Nota che la scelta non è casuale o forzata, ma è pensata, infatti il Paziente ha decisamente molti più accessi e relazioni rispetto al genitore, inoltre i collegamenti del genitore sono strettamente legati al paziente).

In questo modo, soprattutto dato che la struttura finale del Database scelta è gerarchica e non relazionale, legare più pazienti a un singolo genitore è molto più difficile.

Si ritiene comunque che questa modifica non sia impossibile da attuare, in quanto la maggiorparte delle view riguardanti il genitore, come Profilo, Appuntamenti o Monitoraggio sono perfettamente aperte a un cambiamento di questo tipo.

Modifica delle Terapie in corso:

Sebbene non esplicitamente richiesto nei requisiti, una prima concezione delle funzionalità, prevedeva la modifica di esercizi e scenari di gioco in corso d'opera da parte del Logopedista e/o genitore.

L'unica possibilità di cambiamento in corso d'opera rimasta è la possibilità di modificare la data di uno scenario in modo da dare più tempo al bambino nel caso non riesca a raggiungere gli obiettivi sperati nei tempi sperati.

Non sarebbe difficile evolvere l'App al fine di aggiungere questa funzionalità in quanto la funzionalità di Creazione di Terapia, mette già a disposizione elementi molto simili a quelli che si userebbero per la modifica di una terapia già avviata.

1.2.4 Difficoltà Principali dello Sviluppo

Lista e descrizione delle maggiori difficoltà incontrate nello sviluppo e conseguenti soluzioni adottate.

1.2.4.1 Utilizzo di Database NoSQL

Il Database utilizzato per il Progetto è Firebase Realtime Database. Tale Database è un db NoSQL, nello specifico i dati sono rappresentati in maniera gerarchica in grande file formato JSON.

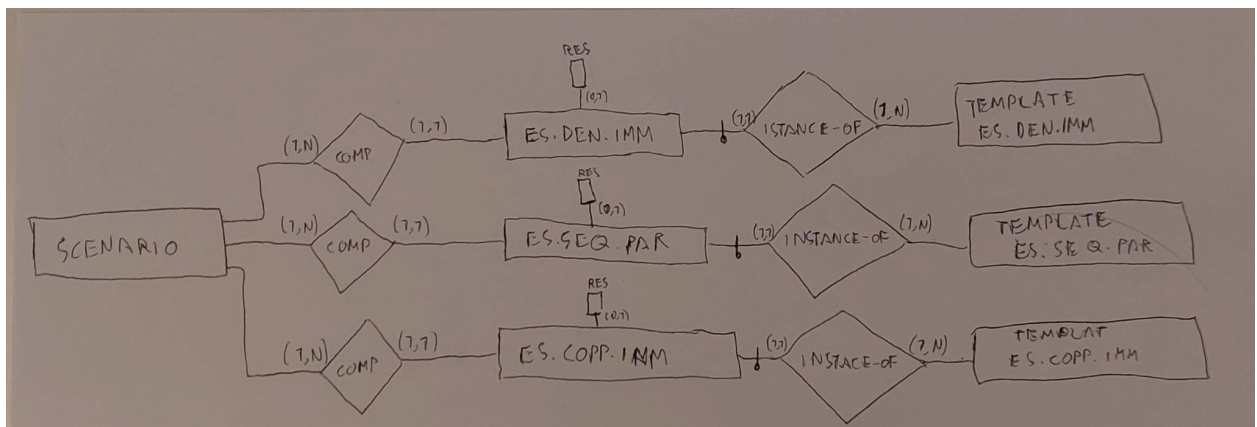
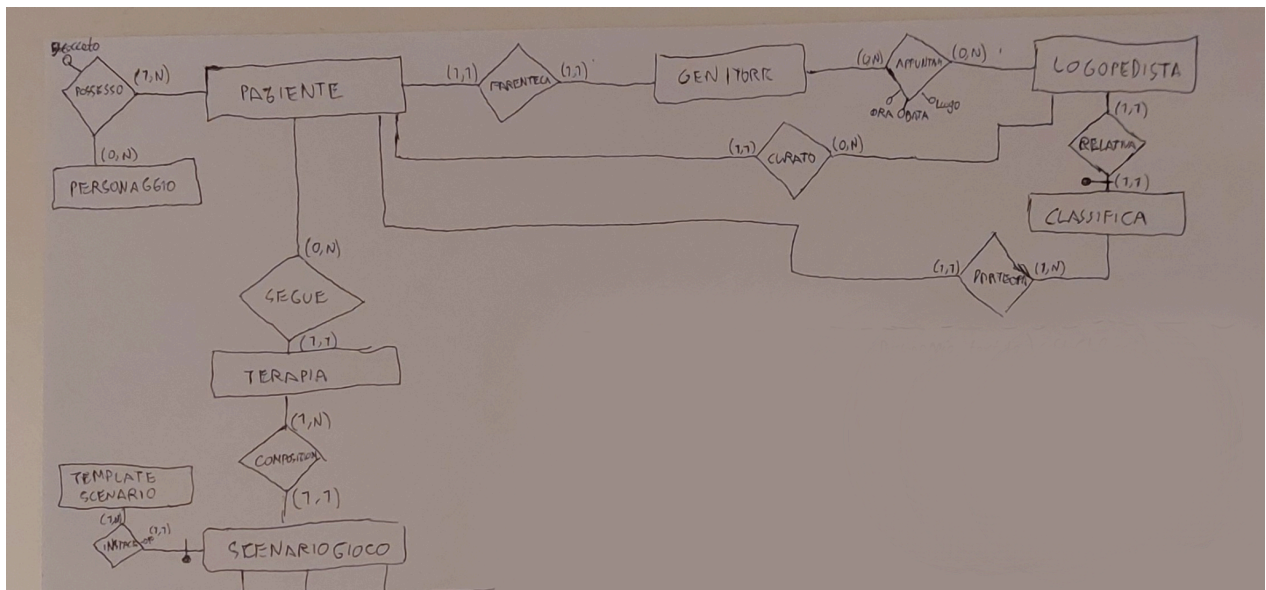
Non avendo esperienza con i Database NoSQL, sono state riscontrate diverse difficoltà. In primo luogo difficoltà nella comprensione del funzionamento e sulla gestione dei dati, queste prime difficoltà sono state rapidamente risolte leggendo approfonditamente la documentazione.

La vera grande difficoltà è stata quella relativa alla strutturazione del Database.

Primo Approccio:

Il primo approccio è stato quello di strutturare i dati nel db simulando un database Relazionale; quindi partendo dal diagramma ER (quello di seguito è parziale, non è stato completato in quanto si è cambiata strutturazione dei dati successivamente) si è convertito il Diagramma Concettuale in Logico, definendo quindi per ogni entità le chiavi primarie e chiavi esterne.

Ovviamente i Vincoli di Chiave Esterne e chiave primaria non erano davvero implementati a livello di DB, le chiavi esterne erano solo riferimenti id, mentre le chiavi primarie erano semplicemente l'identificativo di un nodo.



La struttura del Database derivata da questi Diagrammi ER è stata la seguente:

SCHEMA DB:

```
PERSONAGGIO(idPersonaggio[PK], nomePersonaggio, texture, costoSblocco)
SBLOCCOPERSONAGGIO(idPersonaggio[PK], idPaziente[PK], sbloccato)

PAZIENTE(idProfilo[PK], nome, cognome, username, email, password, eta, dataNascita, sesso, valuta, punteggioTot, idLogopedista, idGenitore)

GENITORE(idProfilo[PK], nome, cognome, username, email, password, telefono, idPaziente)
APPUNTAMENTO(idAppuntamento[PK], idGenitore, idLogopedista, data, ora, luogo)

LOGOPEDISTA(idProfilo[PK], nome, cognome, username, email, password, telefono, indirizzo)
CLASSIFICA(idLogopedista[PK], classificaPazienti)

TERAPIA(idTerapia[PK], dataInizio, dataFine, idPaziente)

TEMPLATE_SCENARIO(idTemplateScenarioGioco[PK], immagineSfondo)
SCENARIOGIOCO(idScenarioGioco[PK], idTemplateScenarioGioco, immagineSfondo, dataInizio, ricompensaFinale, idTerapia)

TEMPLATEESERCIZIODENOMINAZIONEIMMAGINE(idTemplateEsercizio [PK], immagineEsercizio, ricompensaCorretto, ricompensaErrato)
TEMPLATEESERCIZIOSEQUENZAPAROLE(idTemplateEsercizio [PK], audioEsercizio, parola1, parola2, parola3, ricompensaCorretto, ricompensaErrato)
TEMPLATEESERCIZIOCOPPIAIMMAGINI(idTemplateEsercizio [PK], audioEsercizio, immagineEsercizioCorretta, immagineEsercizioErrata, ricompensaCorretto, ricompensaErrato)

ESERCIZIODENOMINAZIONEIMMAGINE(idEsercizio[PK], idTemplateEsercizio, countAiuti, idScenarioGioco)
ESERCIZIOSEQUENZAPAROLE(idEsercizio[PK], idTemplateEsercizio, idScenarioGioco)
ESERCIZIOCOPPIAIMMAGINI(idEsercizio[PK], idTemplateEsercizio, idScenarioGioco)

RISULTATO(idEsercizio[PK], corretto, audioRegistrato)
```

Secondo Approccio:

Alcuni componenti del gruppo, ritenendo sbagliato l'utilizzo di soluzioni sostitutive alle chiavi esterne in un database di tipo NoSQL, hanno spinto per una ristrutturazione del modello dei dati.

L'idea prevedeva utilizzare, per gli elementi lato-codice che contenessero una lista di oggetti, rappresentare sul DB tali nodi come liste di riferimenti a tali oggetti nei nodi corrispondenti.

SCHEMA DB:

```
PERSONAGGI(idPersonaggio[PK], nomePersonaggio, texturePersonaggio, costoSblocco)

PAZIENTI(idProfilo[PK], nome, cognome, username, email, password, eta, dataNascita, sesso, valuta, punteggioTot, listaPersonaggiSBloccati, listaTerapie, idLogopedista)

GENITORI(idProfilo[PK], nome, cognome, username, email, password, telefono, listaAppuntamenti, idPaziente)
APPUNTAMENTI(idAppuntamento[PK], data, ora, luogo)

LOGOPEDISTI(idProfilo[PK], nome, cognome, username, email, password, telefono, indirizzo, listaAppuntamenti, listaPazienti)
CLASSIFICHE(idLogopedista[PK], classificaPazienti)

TERAPIE(idTerapia[PK], dataInizio, dataFine, listaScenariGioco)

TEMPLATE_SCENARIGIOCO(idTemplateScenarioGioco[PK], immagineSfondo)
SCENARIGIOCO(idScenarioGioco[PK], immagineSfondo, dataInizio, ricompensaFinale, listaEsercizi, idTemplateScenarioGioco)

TEMPLATE_ESERCIZIO_DENOMINAZIONE_IMMAGINE(idTemplateEsercizio [PK], immagineEsercizio, ricompensaCorretto, ricompensaErrato)
TEMPLATE_ESERCIZIO_SEQUENZA_PAROLE(idTemplateEsercizio [PK], audioEsercizio, parola1, parola2, parola3, ricompensaCorretto, ricompensaErrato)
TEMPLATE_ESERCIZIO_COPPIA_IMMAGINI(idTemplateEsercizio [PK], audioEsercizio, immagineEsercizioCorretta, immagineEsercizioErrata, ricompensaCorretto, ricompensaErrato)

ESERCIZIO_DENOMINAZIONE_IMMAGINE(idEsercizio[PK], immagineEsercizio, ricompensaCorretto, ricompensaErrato, countAiuti, idTemplateEsercizio)
ESERCIZIO_SEQUENZA_PAROLE(idEsercizio[PK], audioEsercizio, parola1, parola2, parola3, ricompensaCorretto, ricompensaErrato, idTemplateEsercizio)
ESERCIZIO_COPPIA_IMMAGINI(idEsercizio[PK], audioEsercizio, immagineEsercizioCorretta, immagineEsercizioErrata, ricompensaCorretto, ricompensaErrato, idTemplateEsercizio)

RISULTATI_ESERCIZI(idEsercizio[PK], esitoCorretto, audioRegistrato)
```

Il problema di questa strutturazione dei dati nel DB, era la difficoltà di retrieve associata alle liste di riferimenti, le quali avrebbero richiesto l'esecuzione di Query, per giunta asincrone, soltanto per popolare porzioni di oggetti.

Soluzioni Adottate:

Per la serie di problematiche descritte nelle due soluzioni iniziali, si è infine deciso di optare per una strutturazione dei dati gerarchica; come prevede Firebase Realtime Database di base.

Ovviamente non tutti i dati sono stati annidati, ad esempio per gli Appuntamenti (che sono comuni sia ai logopedisti che ai genitori) è stata definito un Nodo a parte con i riferimenti a logopedisti e genitori (ma questo non è l'unico esempio).

```
DB
-TEMPLATE_ESERCIZI
  - (idTemplateEsercizio[PK], ricompensaCorretto, ricompensaErrato)
  - (immagineEsercizio, parolaEsercizio, audioAiuto) OR (audioEsercizio, parola1, parola2, parola3) OR (audioEsercizio, immagineEsercizioCorretta, immagineEsercizioErrata)

-TEMPLATE_SCENARIOGIOCO
  - (idTemplateScenariogioco[PK], immagineSfondo)

-PERSONAGGI
  - (idPersonaggio[PK], nomePersonaggio, texturePersonaggio, costoSblocco)

-APPUNTAMENTI
  - (idAppuntamento[PK], refIdLogopedista, refIdGenitore, data, ora, luogo)

-LOGOPEDISTI
  - (idProfilo[PK], nome, cognome, username, email, password, telefono, indirizzo)
  -CLASSIFICA
    - (classificaPazienti)

-PAZIENTI
  - (idProfilo[PK], nome, cognome, username, email, password, eta, dataNascita, sesso, valuta, punteggioTot, mappaPersonaggiSbloccati)
  -GENITORE
    - (idProfilo[PK], nome, cognome, username, email, password, telefono)
  -TERAPIE
    - (idTerapia[PK], dataInizio, dataFine)
  -SCENARIOGIOCO
    - (idScenariogioco[PK], immagineSfondo, dataInizio, ricompensaFinale, idTemplateScenariogioco)
  -ESERCIZI
    - (idEsercizio[PK], ricompensaCorretto, ricompensaErrato, idTemplateEsercizio)
    - (immagineEsercizio, parolaEsercizio, audioAiuto) OR (audioEsercizio, parola1, parola2, parola3) OR (audioEsercizio, immagineEsercizioCorretta, immagineEsercizioErrata)
  -RISULTATO
    - (esitoCorretto)
    - (countAiuti) OR (audioRegistrato) OR (audioRegistrato)
```

La versione finale del DB verrà posta nella sezione relativa alla Architettura del Sistema.

1.2.4.2 Asincronicità delle Operazioni su Database

La più grande difficoltà nello sviluppo del progetto è stata la gestione della Asincronicità delle operazioni su Database.

In particolare le operazioni più problematiche erano quelle di retrieve (get) dei dati.

In genere i Database in locale, come, nel caso di Android, SQLite, permettono di eseguire le operazioni sul Database sempre in modo sincrono; per i Database online, come Firebase, questo non avviene, e la maggiorparte delle operazioni effettuate sulle classi di Firebase sono operazioni asincrone.

Nel progetto sono state utilizzate delle classi DAO per la gestione delle operazioni su DB; ragionando in modo sincrono, i client delle classi DAO, chiamano i metodi get e ottengono il valore necessario per lo svolgimento delle loro operazioni. In un contesto asincrono non si può ragionare in questo modo, in quanto la funzione asincrona ritornerà il risultato ottenuto dall'esecuzione della porzione sincrona di codice, restituendo il vero risultato asincrono solo dopo la chiamata da parte della classe client.

Normalmente, ad esempio in semplice codice java, questo potrebbe essere risolto semplicemente con meccanismi quali Thread, Wait, Sleep, Latch e altri costrutti per “aspettare” il completamento dell'operazione asincrona in modo da ottenere sincronia. Questo su Android non è possibile, in quanto, Android, in maniera trasparente a Java, pare gestire autonomamente le operazioni asincrone, e questo comportamento probabilmente è legato alla gestione del ciclo di vita delle Activity su cui sta effettivamente avvenendo il set di operazioni.

Primo Approccio:

Il primo metodo per risolvere questo problema è stato pensare a delle “Interfacce di Callback”.

In pratica veniva definita una interfaccia, contenente metodi del tipo “onSuccess()” o “onFailure”, implementata da tutte le classi DAO o comunque da tutte le classi con operazioni asincrone.

I metodi asincroni dovevano quindi prendere in Input un oggetto di tipo callback, e alla chiamata da parte dei client del metodo, questi dovevano fornire una implementazione al callback; nello specifico dovevano definire le azioni da eseguire una volta completata l'esecuzione del metodo asincrono.

Soluzioni Adottate:

La soluzione finale adottata per la gestione delle operazioni asincrone su Database è stata quella degli oggetti di tipo `CompletableFuture<>`.

Un oggetto `CompletableFuture` permette tramite la chiamata a metodi `.supplyAsync()` o `.completeFuture()`, di eseguire operazioni in background. Successivamente la classe client che chiama il metodo che restituisce un `CompletableFuture`, dovrà chiamare sul `CompletableFuture` il metodo `.thenAccept`, il quale prende in input un predicato (lambda) dove vanno definite le operazioni da eseguire al termine della operazione asincrona (completamento del Future).

Questa soluzione rispetto alla precedente è più pulita, in quanto lascia praticamente invariate le firme dei metodi asincroni (l'unica differenza di tali metodi è quella che anziché dover restituire Type, restituiranno `CompletableFuture<Type>`).

Sono stati adottati all'interno del progetto due modi principali per utilizzare il Future; a livello di classi client non vi è differenza, differenza è nei metodi asincroni.

Primo metodo: in questo primo metodo si dichiara un `CompletableFuture` senza iniziarlo, poi al termine delle operazioni asincrone, si restituisce il risultato della operazione avvalorando il Future. Si può anche avvalorare il Future con una eccezione nel caso di fallimento di operazione asincrona. Infine viene ritornata il Future.

```
public CompletableFuture<List<Personaggio>> get(String field, Object value) {
    CompletableFuture<List<Personaggio>> future = new CompletableFuture<>();

    DatabaseReference ref = db.getReference(CostantiNodiDB.PERSONAGGI);
    if (true /*operazione ha successo*/) {
        future.complete(null /*risultato*/);
    }
    else { //operazione non ha successo
        future.completeExceptionally(/*errore nella operazione asincrona*/ new Exception());
    }

    return future;
}
```

Secondo metodo: in questo secondo metodo si utilizza `supplyAsync()`. All'interno delle operazioni da eseguire è necessaria una istruzione di `wait`, come ad esempio un `while(!task.isComplete())`. Per quanto detto in precedenza, in realtà il flusso di esecuzione skipa subito il `while` senza aspettare il ritorno dal Database, tuttavia se si utilizza il metodo `.thenAccept()` nella client chiamante, il Future verrà completato prima della esecuzione delle operazioni interne al `.thenAccept()`. Questa soluzione è più leggibile della precedente ma meno controllabile.

```
public CompletableFuture<Personaggio> getById(String idObj) {
    return CompletableFuture.supplyAsync(() -> {
        DatabaseReference ref = db.getReference(CostantiNodiDB.PERSONAGGI).child(idObj);
        Task<DataSnapshot> task = ref.get();

        Personaggio result = null;

        while (!task.isComplete()) {}

        DataSnapshot snapshot = task.getResult();
        Map<String, Object> fromDatabaseMap = (Map<String, Object>) snapshot.getValue();
        result = new Personaggio(fromDatabaseMap, idObj);

        Log.d(tag: "PersonaggioDAO.getById()", result.toString());
        return result;
    });
}
```

1.2.4.3 Asincronicità di alcune Operazioni Android

In genere, oltre alle operazioni dette in precedenza, ovvero quelle su Database, nella programmazione dell'Applicazione ci si è imbattuti in diverse operazioni asincrone.

Esempi sono le Animazioni, Dialoghi, Tooltip, Autenticazione e soprattutto le operazioni inerenti alla gestione degli Audio.

È un tipo di problematica praticamente identica a quella detta in precedenza riguardante il Database.

Soluzioni Adottate:

Le soluzioni adottate sono praticamente analoghe a quelle descritte in precedenza.

Si sono utilizzati gli oggetti `CompletableFuture` come return per le operazioni asincrone, e il metodo `.thenAccept()` per la gestione delle operazioni da eseguire dopo la terminazione del metodo asincrono.

1.2.4.4 Classi ViewModel

Il più grande problema legato alla scelta del Pattern Architetturale MVVM, Model View-ViewModel (spiegato nella sezione relativa alla Architettura), è stata la comprensione del significato e funzionamento della macrocomponente ViewModel.

In pratica nel MVVM, i ViewModel devono essere istanziati dalle View che li utilizzano per eseguire operazioni e aggiornare i dati da visualizzare.

La comprensione di ciò è stata già una prima difficoltà; infatti si ragionava ancora con il familiare Model-View-Controller, il quale è leggermente diverso, in quanto le View sono responsabili della loro stessa gestione piuttosto che i ViewModel.

Il vero problema ha riguardato la gestione delle istanze di oggetti di Business. Infatti in ognuna delle Activity (Logopedista, Genitore, Paziente), è necessario avere la istanza del tipo di utente che ha effettuato l'accesso alla applicazione, per svolgere le funzionalità dell'App messe a disposizione dai vari Fragment.

Inizialmente si è pensato, ancora ragionando in modo semplice, di mappare 1 a 1 le classi tra View e ViewModel, e quindi dare un ViewModel per ogni Fragment. Questa soluzione non è del tutto sbagliata ma richiede l'implementazione della comunicazione tra ViewModel, la quale è difficile non solo da implementare ma anche da comprendere.

Possibili Soluzioni:

La prima soluzione possibile era quella di procedere con la creazione di un ViewModel per ogni Fragment, in questo caso quindi ogni Fragment, attraverso il ViewModel, avrebbe accesso ai dati contenuti all'interno del proprio ViewModel.

Questa soluzione richiederebbe l'implementazione della comunicazione tra ViewModel, la quale è possibile solo attraverso i cosiddetti ViewModel Factory.

Questa soluzione non solo è sembrata difficile da implementare e astrarre, ma anche da comprendere.

Soluzioni Adottate:

La soluzione adottata è una soluzione meno potente rispetto a quella ottima dell'utilizzo dei Factory per la comunicazione tra ViewModel, ma è comunque una soluzione ottimale, totalmente in linea con il pattern MVVM.

Si è deciso di implementare i ViewModel solo per le Activity. Quindi solo le Activity hanno i propri ViewModel, i quali posseggono effettivamente la istanza dell'oggetto che rappresenta l'utente loggato e i suoi dati.

Tutti i Fragment relativi alla Activity, istanziano il ViewModel mediante il concetto di "Shared ViewModel", dove in pratica si inizializza il ViewModel passando come parametro la Activity a cui il Fragment fa riferimento piuttosto che il Fragment stesso.

Inoltre ogni Fragment possiede un proprio Controller, il quale è istanziato dentro il Fragment e permette al Fragment di compiere tutte quelle azioni di gestione che non riguardano la View. Il ViewModel delle Activity possiede un attributo per ogni Controller che vi fa riferimento.

Esempio - ViewModel della Activity:

```
9 usages  FilippoChinniUniba
public class LogopedistaViewModel extends ViewModel {
    4 usages
    private Logopedista mLogopedista;
    3 usages
    private RegistrazionePazienteGenitoreController mRegistrazionePazienteGenitoreController;

    FilippoChinniUniba
    public Logopedista getLogopedista() { return mLogopedista; }
    FilippoChinniUniba
    public void setLogopedista(Logopedista logopedista) { mLogopedista = logopedista; }

    2 usages  FilippoChinniUniba
    public void aggiornaLogopedistaRemoto() {
        LogopedistaDAO logopedistaDAO = new LogopedistaDAO();
        logopedistaDAO.update(mLogopedista);
        Log.d( tag: "LogopedistaViewModel.aggiornaLogopedistaRemoto()", msg: "Logopedista aggiornato: " + mLogopedista.toString());
    }

    1 usage  FilippoChinniUniba
    public RegistrazionePazienteGenitoreController getRegistrazionePazienteGenitoreController() {
        if (this.mRegistrazionePazienteGenitoreController == null) {
            this.mRegistrazionePazienteGenitoreController = new RegistrazionePazienteGenitoreController();
        }
        return this.mRegistrazionePazienteGenitoreController;
    }
}
```

Esempio - Fragment:

```
4 usages
private LogopedistaViewModel mLogopedistaViewModel;
6 usages
private RegistrazionePazienteGenitoreController mController;

FilippoChinniUniba +1
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {

    View view = inflater.inflate(R.layout.fragment_registrazione_paziente_genitore, container, attachToRoot: false);

    this.mLogopedistaViewModel = new ViewModelProvider(requireActivity()).get(LogopedistaViewModel.class);
    this.mController = mLogopedistaViewModel.getRegistrazionePazienteGenitoreController();
}
```

Esempio - Controller del Fragment:

```
1 usage  FilippoChinniUniba
public Genitore registrazioneGenitore(String userId, String nome, String cognome, String username, St
    TipoUtente tipoUtente = TipoUtente.GENITORE;

    Genitore genitore = new Genitore(userId, nome, cognome, username, email, password, telefono);
    GenitoreDAO genitoreDAO = new GenitoreDAO();
    genitoreDAO.save(genitore, idLogopedista, idPaziente);

    helperRegistrazione(userId, tipoUtente);

    return genitore;
}

1 usage  FilippoChinniUniba
public Paziente registrazionePaziente(String userId, String nome, String cognome, String username, St
    TipoUtente tipoUtente = TipoUtente.PAZIENTE;

    Paziente paziente = new Paziente(userId, nome, cognome, username, email, password, eta, dataNasci
    PazienteDAO pazienteDAO = new PazienteDAO();
    pazienteDAO.save(paziente, idLogopedista);

    helperRegistrazione(userId, tipoUtente);

    return paziente;
}
```

1.2.4.5 Tempistiche del Progetto

Un problema persistente durante l'intero processo di sviluppo, è stata la gestione del tempo.

Nonostante una pianificazione ex-Ante delle tempistiche del progetto, facendo tutti i 3 altri esami al primo appello (compreso esonero), per quindi potersi dedicare totalmente allo sviluppo di questo progetto, si sono comunque trovate difficoltà.

Per limitare il numero di refactor, si è pensato sin dall'inizio di utilizzare procedure rigorose, definendo prima i requisiti, poi l'architettura del sistema, servendosi fin da subito di strumenti come Diagramma delle Classi e Diagrammi ER.

La prima fase ha anche compreso lo studio del pattern architetturale. Tutte queste azioni iniziali definite, avevano lo scopo, e hanno in buona parte anche raggiunto il risultato, di limitare il costo degli errori durante lo sviluppo (più tardi si commette l'errore peggio è, più si definisce bene all'inizio e meno costerà risolvere).

Nonostante tutto questo, ci si è trovati in una condizione di forte pressione durante tutto il periodo dello sviluppo, lavorando praticamente tutto il giorno, con pause solo a pranzo e cena; per alcuni membri del gruppo anche di notte (questo paragrafo sta venendo scritto alle 02:21).

Le cause principali del problema non riguardavano soltanto difficoltà e ampiezza dei requisiti, i quali tutto sommato erano gestibili, quanto la mole di lavoro necessaria per imparare nuovi strumenti (a livello di codice) per la programmazione in Android (come programmazione a eventi, threading e asincronicità), e gli eccessivi costi di refactoring dovuti a cattive comprensioni dell'utilizzo di costrutti o della strutturazione del DB.

Soluzioni Adottate:

Non ci sono state effettivamente vere soluzioni; si è dormito di meno, lavorato di più, e sofferto in silenzio.

(Questo Paragrafo è stato scritto prima della proroga della consegna)

ARCHITETTURA DEL SISTEMA

PRONUNTIAPP

2. ARCHITETTURA DEL SISTEMA

2.1 Architettura di Alto Livello del Sistema

Il pattern architetturale scelto per lo sviluppo del progetto è il pattern Model-View-ViewModel.

Il pattern Model-View-ViewModel permette di realizzare la separazione degli interessi tra le 3 principali macrocomponenti del codice sorgente di un progetto Android.

Il Model contiene la definizione delle Entità di Business, e classi e metodi per l'accesso ai dati (come ad esempio la persistenza). Inoltre si collega alle classi della infrastruttura esterna.

Il ViewModel si comporta come mediatore tra Model e View. Fornisce alla View dei metodi per ottenere i dati da visualizzare provenienti dal Model.

La View è l'interfaccia utente della applicazione. Si compone delle Activity, dei Fragment e la gestione di altri oggetti grafici più complessi.

Implementazione del Model-View-ViewModel:

Il progetto (escludendo la cartelle res, assets e la cartella di Gradle), è costituito da 3 macrocomponenti, package principali.

Il package models contiene:

- autenticazione: classi per l'Autenticazione di Firebase
- database: classi DAO per la comunicazione con il Database Firebase e costanti del DB
- domain: definizione di tutte le classi di Entità di Business
- external_api: classi per la gestione delle librerie esterne, in particolare quelle che necessitano di astrazione
- utils: classi per la gestione di Audio Recorder e Audio Player

Il package viewmodels contiene:

- autenticazione_viewmodel: viewmodel e gestori relativi a Login e Registrazione
- logopedista_viewmodel: viewmodel della activity del Logopedista, e controller per i fragment ad essa associati
- paziente_viewmodel: viewmodel della activity del Paziente, e controller per i fragment ad essa associati
- genitore_viewmodel: viewmodel della activity del Genitore, e controller per i fragment ad essa associati

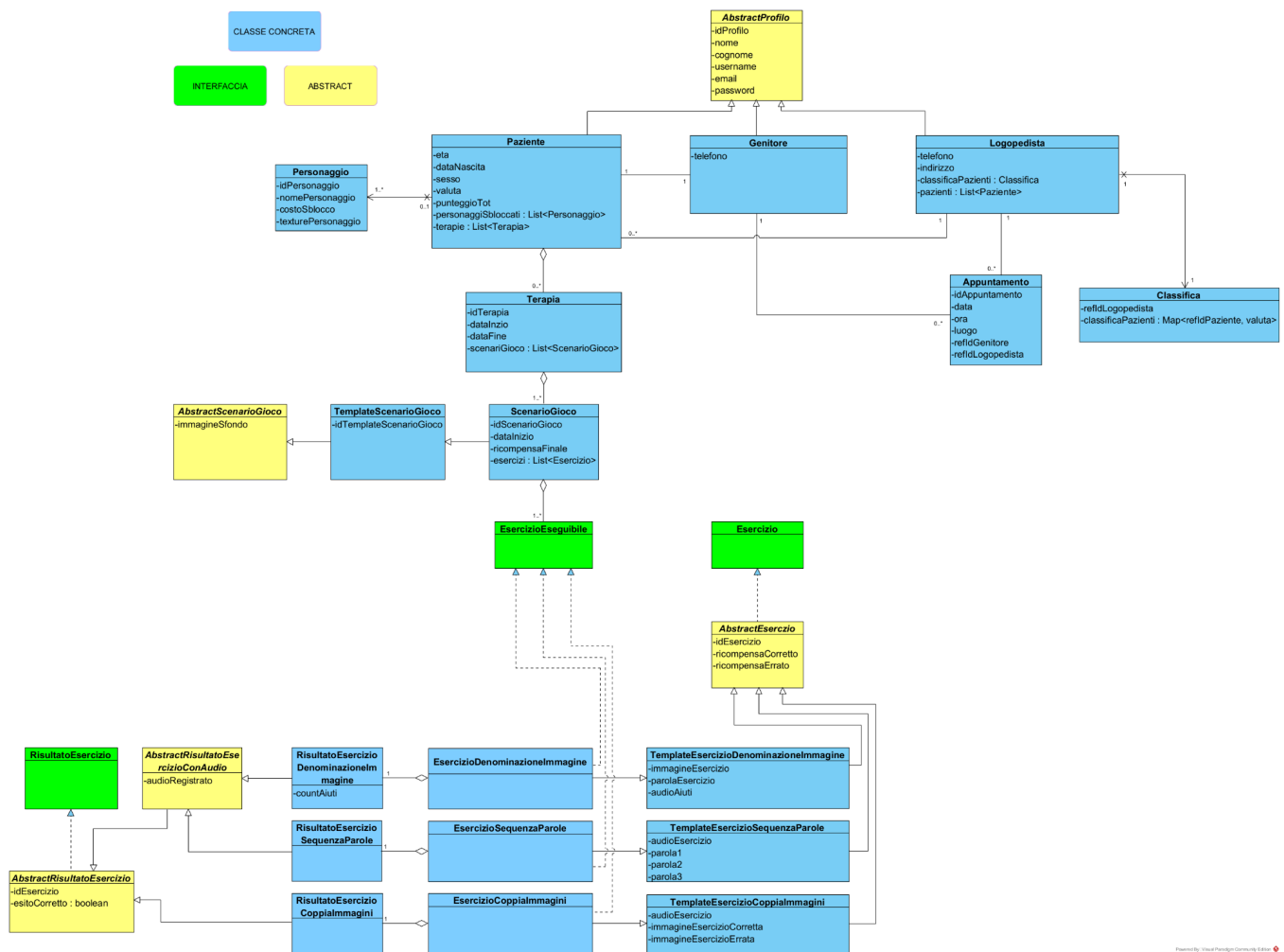
Il package views contiene:

- activity: definizione delle Activity dell'App
- fragment: definizione dei Fragment dell'App
- dialog: definizione dei Dialog
- navigation_selection: classi per la gestione della navBar

2.2 Diagramma delle Classi

Diagramma delle Classi della Applicazione (solo Model).

Utile per descrivere il dataflow del programma, le dipendenze tra le classi (e i dati), e le relazioni di ereditarietà.

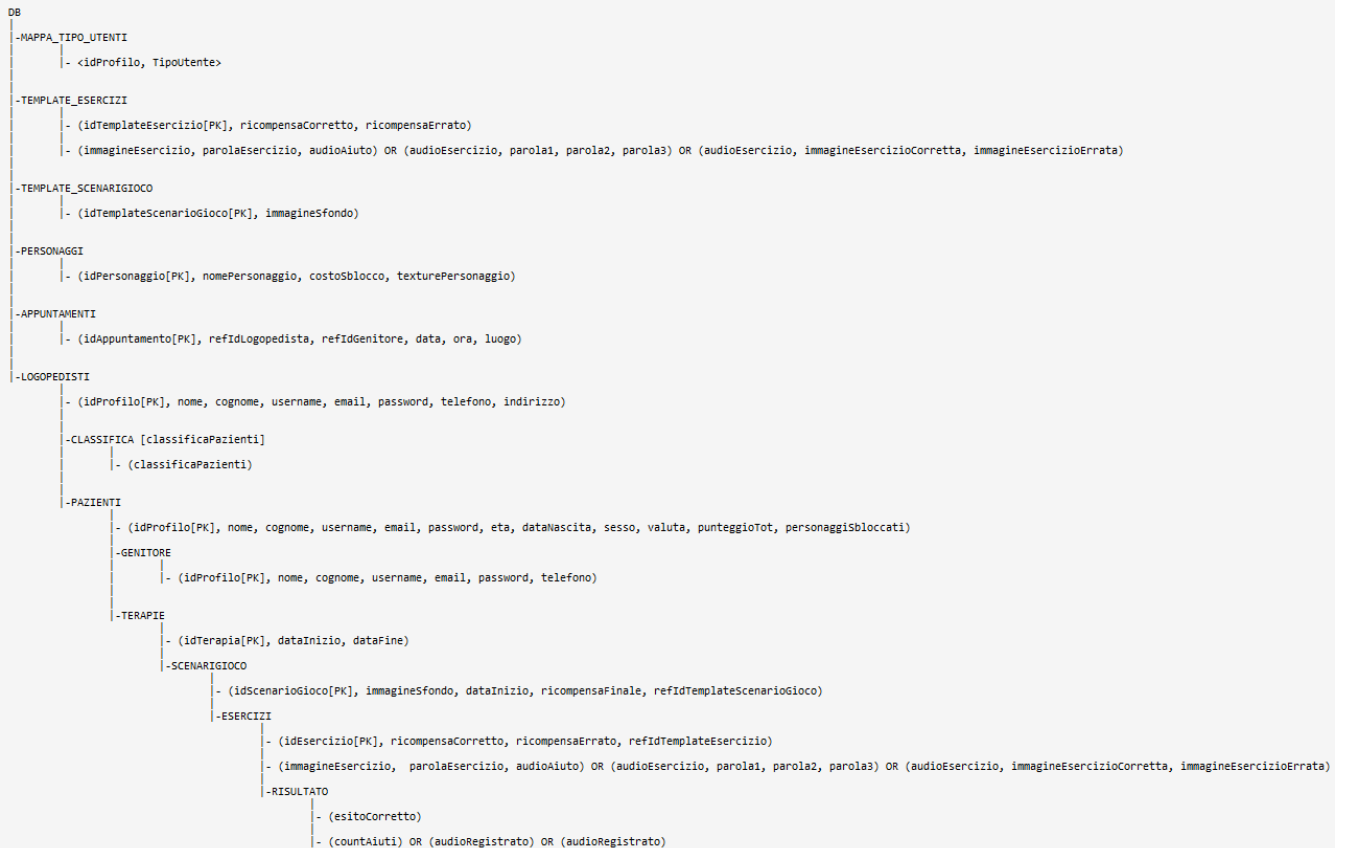


Si omette in questo Diagramma delle classi, che tutte le classi visibili implementano un'interfaccia "Persistente" (che in fornisce metodi toMap() e fromMap() rispettivamente per formattare gli oggetti to e from il database).

Tale classe Persistente implementa a sua volta Serializable per rendere possibile passare tutti gli oggetti tramite extra degli Intent.

2.3 Gerarchia dei Dati

In quanto Firebase è un Database NoSQL, in particolare uno dove i dati sono raggruppati in modo gerarchico, la rappresentazione finale della struttura dei dati è gerarchica.



In particolare, si fanno le seguenti note:

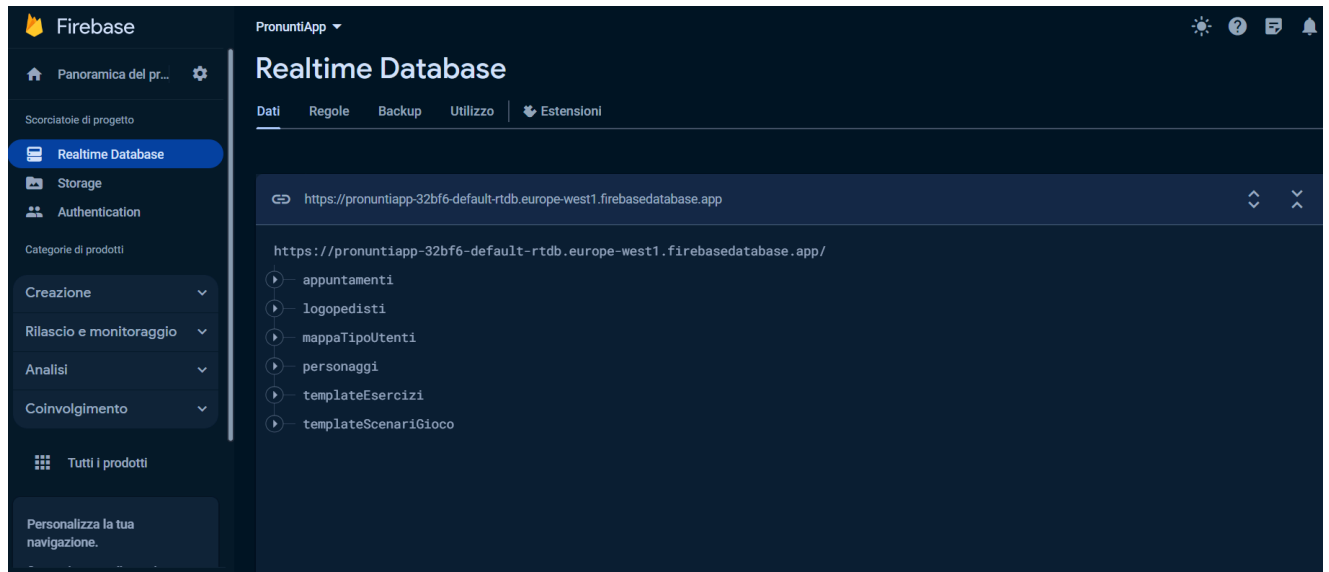
- I nodi principali (1° livello) hanno tutti un Id univoco, generato da Firebase e assegnato al momento della creazione.
- I nodi rappresentanti gli utenti (Logopedista, Genitore, Paziente), sono identificati con lo UserId fornito da FirebaseAuthentication al momento della loro registrazione nel sistema.
- Gli altri nodi sono identificati con id sequenziali relativi, dove in pratica il numero indica l'indice della lista di oggetti all'interno della quale si trovano.

2.3.1 Cruscotto Firebase

Si allega la Dashboard di Firebase come richiesto:

ABCD

software developing



RUOLI DEL TEAM

PRONUNTIAPP

3. RUOLI DEL TEAM

3.1 Struttura del Team

Il team di sviluppo di PronuntiApp è formato da 4 membri:

- Filippo Chinni Carella
- Pasquale Di Gennaro
- Diego Amati
- Nicola Balzano

Il leader del gruppo è Filippo Chinni Carella.

3.2 Suddivisione

Ripartizione dei compiti e delle funzionalità sviluppate da ogni membro del team.

La suddivisione non è assoluta, nel senso che per alcune funzionalità si è lavorato in coppia o in gruppo, e in generale i membri del team hanno eseguito una revisione reciproca dei propri manufatti realizzati.

Dove è scritto (**parzialmente**) si intende che il membro ha lavorato a quel componente, ma non ne è l'artefice principale.

Se invece una stessa componente compare 2 o più volte su membri diversi senza indicazione, allora significa che hanno lavorato insieme alla pari su tale funzionalità.

3.2.1 Filippo Chinni Carella

Funzionalità e mansioni svolte nel progetto da Filippo Chinni Carella.

Progettazione Concettuale:

- Interpretazione dei Requisiti
- Concezione delle funzionalità
- Concezione del dataflow del sistema

Progettazione Architettuale:

- Definizione dell'Architettura del Sistema
- Diagramma delle Classi
- Diagrammi ER
- Definizione dello Schema del Database
- Definizione della Struttura Gerarchica e dei Nodi del Database

Package models:

- Package database
 - Definizione delle classi DAO
 - Definizione delle classi di Costanti del DB
 - Definizione dei metodi per le operazioni CRUD fondamentali su Database

- Definizione di metodi per il Read dei dati più specifici
- Definizione dei metodi per la persistenza dei dati multimediali (Firebase Storage)
- Package dominio
 - Definizione delle classi di Entità di Business
 - Scrittura dei metodi toMap() e fromMap() per ogni Entità
- Package risorse
 - Definizione dei messaggi di errore (Log)
 - Definizione dei messaggi di avviso (Log)
- Package Autenticazione
 - Collegamento con Firebase Authentication
 - Definizione di classe e metodi per il Login e la Registrazione tramite Firebase Authentication
- Package external_api
 - Refactoring, ottimizzazione e astrazione
 - Implementazione di AudioRecorder (parzialmente)
 - Implementazione di AudioPlayer (parzialmente)

Package viewmodels:

- Definizione generale della struttura del package
- Package Autenticazione_ViewModel
 - LoginViewModel
 - RegistrazioneViewModel
- Package Logopedista_ViewModel
 - Definizione della classe principale LogopedistaViewModel
 - RegistrazionePazienteGenitoreController (parzialmente)
 - ModificaAppuntamentiController
- Package Genitore_ViewModel
 - Definizione della classe principale GenitoreViewModel
- Package Paziente_ViewModel
 - Definizione della classe principale PazienteViewModel
 - EsercizioDenominazioneImmagineController
 - EsercizioSequenzaParoleController
 - EsercizioCoppiaImmaginiController
 - ClassificaController (parzialmente)
 - PersonaggiController
 - MonitoraggioController (parzialmente)
- Implementazione di altri vari Controller

Package views:

- Definizione della Logica di Inizializzazione delle Activity
- Implementazione dei Controller di Inizializzazione (init_controllers)
- Package activity
 - AutenticazioneActivity
 - EntryActivity
 - LogopedistaActivity (parzialmente)
 - GenitoreActivity (parzialmente)
 - PazienteActivity (parzialmente)

- Package dialog
 - Definizione Generale dei Dialog (parzialmente)
 - AbstractPopUpDialog (parzialmente)
 - InfoDialog
 - ConnessioneMancanteDialog
- Package fragment
 - AvvioRapidoFragment (solo Backend)
 - LoginFragment (solo Backend)
 - RegistrazioneFragment (solo Backend)
 - LogoutFragment
 - AppuntamentiLogopedistaFragment (parzialmente)
 - RegistrazionePazienteGenitoreFragment (parzialmente)
 - EsercizioDenominazioneImmagineFragment (parzialmente)
 - EsercizioSequenzaParoleFragment (parzialmente)
 - EsercizioCoppiaImmaginiFragment (parzialmente)
 - MonitoraggioFragment (parzialmente)
 - CreazioneTerapiaFragment
- Revisioni varie

Cartella res:

- Definizione dei layout di Fragment (parzialmente)
- Definizione dei values (parzialmente)
 - string eng (parzialmente)
 - string ita (parzialmente)
- Revisioni Varie

Funzionalità Principali:

- Login
- Registrazione
- Esercizio Denominazione Immagine (parzialmente)
- Esercizio Sequenza Parole (parzialmente)
- Esercizio Coppia Immagini (parzialmente)
- Creazione e visualizzazione degli Appuntamenti (parzialmente)
- Gestione dei Permessi
- Gestione delle Connessione Mancante
- Gestione Schermate di Caricamento

Testing:

- Definizione di un Package temporaneo per il Testing
- Definizione di Activity e Fragment per il Testing

Progettazione dell'Icona:

- Documentazione di Progettazione Icona
- Progettazione Icona
- Realizzazione Icona (parzialmente)

Documentazione:

- Definizione della struttura delle 3 Documentazioni

- Documentazione di Sviluppo
- Documentazione di Manuale Utente

Misc:

- Implementazione Verifica Connessione Mancante
- Implementazione Gestione Permessi
- Implementazione Schermate di Caricamento
- Volantino Pubblicitario

3.2.2 Pasquale Di Gennaro

Funzionalità e mansioni svolte nel progetto da Pasquale Di Gennaro.

Connessione con il Database:

- Collegamento con il Database (Firebase Realtime Database)
- Inizializzazione del Database (lato server) (Firebase Realtime Database)
- Collegamento a Firebase Storage
- Inizializzazione del Database (lato server) (Firebase Storage)

Package models:

- Package external_api
 - Implementazione del Riconoscimento Vocale
 - Implementazione della Conversione Audio
 - Implementazione di AudioRecorder
 - Implementazione di AudioPlayer

Package viewmodels:

- Package Logopedista_ViewModel
- Package Genitore_ViewModel
- Package Paziente_ViewModel
 - ClassificaController

Package views:

- Package fragment
 - ClassificaFragment (parzialmente)
 - EsercizioDenominazioneImmagineFragment
 - EsercizioSequenzaParoleFragment
 - EsercizioCoppiaImmaginiFragment
 - PersonaggiFragment
 - ScenarioFragment
 - ScenariFragment
 - CreazioneEsercizioFragment
 - CreazioneScenarioFragment
 - CreazioneTerpiaFragment

Funzionalità Principali:

- Esercizio Denominazione Immagine
- Esercizio Sequenza Parole
- Gestione dei Personaggi
- Creazione Terapie

Documentazione:

- Documentazione di Manuale Utente (parzialmente)

Misc:

- Implementazione Gestione Permessi (parzialmente)

3.2.3 Diego Amati

Funzionalità e mansioni svolte nel progetto da Diego Amati.

Package models:

- Definizione classi dominio (parzialmente)
- Getter delle classi dominio
- Setter delle classi dominio
- toString() delle classi dominio

Package viewmodels:

- Package Autenticazione_ViewModel
 - LoginViewModel (parzialmente)
 - RegistrazioneViewModel (parzialmente)
- Package Logopedista_ViewModel
 - RegistrazionePazienteGenitoreController
 - ModificaAppuntamentiController
 - ModificaDataScenariController
- Package Genitore_ViewModel
 - AppuntamentiGenitoreController
 - ModificaDataScenariController

Package views:

- Package fragment
 - LoginFragment (parzialmente)
 - RegistrazioneFragment (parzialmente)
 - ProfiloLogopedistaFragment (parzialmente)
 - ProfiloGenitoreFragment (parzialmente)
 - ProfiloPazienteFragment (parzialmente)
 - AppuntamentiGenitoreFragment
 - AppuntamentiLogopedistaFragment
 - RegistrazionePazienteGenitoreFragment
 - ClassificaFragment (parzialmente)
 - PazientiFragment (Lista dei Pazienti)
 - RisultatoEserciziCoppiaImmaginiFragment
 - RisultatoEserciziDenominazioneParoleFragment
 - RisultatoEserciziSequenzaParoleFragment

- TerapieGenitoreFragment
- TerapieLogopedistaFragment

Funzionalità Principali:

- Creazione e visualizzazione degli Appuntamenti
- Monitoraggio Terapia (lato Genitore)
- Monitoraggio Terapia (lato Paziente)

Documentazione:

- Documentazione di Manuale Utente (parzialmente)

3.2.4 Nicola Balzano

Funzionalità e mansioni svolte nel progetto da Nicola Balzano.

Progettazione Concettuale:

- Interpretazione dei Requisiti (parzialmente)
- Concezione delle funzionalità (parzialmente)

Package viewmodels:

- Definizione generale della struttura del package (parzialmente)

Package views:

- Package activity
 - AbstractAppActivity
 - LogopedistaActivity
 - GenitoreActivity
 - PazienteActivity
 -
- Package dialog
 - Definizione Generale dei Dialog
 - AbstractPopUpDialog
 - Permessi Dialog
 - RichiestaConfermaDialog
- Package fragment
 - AvvioRapidoFragment (solo Frontend)
 - LoginFragment (solo Frontend)
 - RegistrazioneFragment (solo Frontend)
 - ProfiloLogopedistaFragment
 - ProfiloGenitoreFragment
 - ProfiloPazienteFragment
 - AppuntamentiGenitoreFragment
 - AppuntamentiLogopedistaFragment
 - PazientiFragment (solo Frontend)
 - EsercizioDenominazioneImmagineFragment (parzialmente)
 - EsercizioSequenzaParoleFragment (parzialmente)
 - EsercizioCoppiaImmaginiFragment (parzialmente)
 - ClassificaFragment

- ScenarioFragment
- ScenariFragment
- MonitoraggioFragment
- Package navigation_selector
 - Definizione delle NavBar dell'applicazione

Cartella res:

- Definizione dei layout delle Activity
- Definizione dei layout di Fragment
- Definizione dei Menu
- Definizione dei values
 - string eng
 - string ita
 - colors

Funzionalità Principali:

- Scenario di Gioco
- Esercizio Coppia Immagini
- Visualizzazione Profilo
- Creazione e visualizzazione degli Appuntamenti (parzialmente)
- Gestione della Classifica
- Monitoraggio Terapia

Progettazione dell'Icona:

- Progettazione Icona
- Realizzazione Icona

Misc:

- Volantino Pubblicitario